2 Twofold

PRODUCT REQUIREMENTS DOCUMENT

Twofold

A ledger-grade digital wallet for Ethiopia — send, receive, top up, and cash out, with a double-entry ledger that proves every balance.

— Every birr, recorded twice.

Version 1.0 | 2 November 2026 | Hosama Adem

> **Every birr, recorded twice.** A ledger-grade digital wallet for Ethiopia: send, receive, top up, and cash out, with a double-entry ledger underneath that makes it impossible to create or lose money by accident.

Status	v1.0, MVP engineering blueprint
Date	2 November 2026
Owner	Hosama Adem
Repo	`hosama-adem/realledger` (rename to `twofold` planned)
Companion docs	Implementation plan · Design system
Previous product	RealLedger, archived in `docs/archive/realledger`

> **Sandbox only.** Twofold is an engineering portfolio project. It does not hold real money, is not licensed by the National Bank of Ethiopia (NBE), and talks to *simulated* providers. Limits and fees here are illustrative, not regulatory figures.

Table of contents

1. Why Twofold exists
2. Name, positioning, and principles
3. Users and personas
4. Scope
5. Core user journeys
6. Functional requirements
7. The ledger
8. Correctness under concurrency
9. Idempotency
10. State machines
11. Asynchronous work: outbox, jobs, webhooks
12. Risk and limits
13. Admin console
14. Data model
15. API overview
16. Architecture
17. Non-functional requirements

18. Security and privacy
19. Observability
20. Testing strategy
21. Frontend requirements
22. Success metrics
23. Risks and mitigations
24. What this project teaches
25. Credits and prior art
26. Glossary

---

1. Why Twofold exists

1.1 The problem

Most money in Ethiopia now moves digitally. Behind every wallet sits the same set of hard engineering problems:

What happens in real life	What breaks without careful engineering
Two people send money to each other at the same instant	One write overwrites the other; a balance is wrong, or the database deadlocks
A phone loses signal right after tapping **Send**, and the app retries	The money moves twice
A provider callback arrives late, twice, or never	A top-up is stuck in "pending" forever, or credited twice
Someone edits a balance directly in the database	Nobody can prove what happened or when
Fees change between the preview screen and the confirm tap	The user is charged an amount they never agreed to

Most student wallet projects store `balance` in a column and run `UPDATE wallets SET balance = balance - 100`. That works on a laptop and breaks in production.

1.2 What Twofold is

A digital wallet platform where a user can:

1. Sign up with a phone number and get an ETB wallet.
2. Top up from a simulated mobile-money or gateway provider (Telebirr-like, Chapa-like).
3. Send money to another user by phone number or `@handle`.
4. Request money from someone, and pay requests they receive.

5. Pay a merchant by scanning a QR code.
6. Cash out to a simulated bank or mobile-money account.
7. See every movement, with a receipt, forever.

Underneath, a **double-entry ledger** records every movement as balanced debits and credits. Balances come from the ledger. Nobody edits a balance by hand.

1.3 What Twofold is not

- Not a licensed payment service. No real money moves.
- Not a bank: no lending, interest, savings products, or cards.
- No foreign exchange in the MVP. A wallet holds exactly one currency.
- No real provider credentials. Providers are simulated adapters shaped like the real ones.

---

2. Name, positioning, and principles

2.1 Name

Twofold. Every movement is written twice, once as a debit and once as a credit. The name *is* the core guarantee.

- Product tagline: **Every birr, recorded twice.**
- Engineering tagline: **Money that always sums to zero.**

Names that were checked and rejected because Ethiopian finance products already use them: Kelal (Kelal Gateway wallet), Abol (Bunna Bank app), Mezgeb (two finance apps), Mizan (Mizan SACCO wallet and others), BirrLink, Wallet Birr. Run a trademark and domain check on "Twofold" before using it publicly.

2.2 Positioning

> Telebirr shows you a balance. **Twofold shows you why it is right.**

For a reviewer or interviewer, Twofold demonstrates the parts of a payments system that are hard to fake: ledger design, concurrency, idempotency, asynchronous settlement, reconciliation, and auditability. The UI is polished, but the backend is the reason the project exists.

2.3 Product principles

1. **Correct before fast.** A slow transfer is a bug. A wrong balance is an incident.
2. **The ledger is the truth.** Every balance can be rebuilt from postings.
3. **Nothing is edited, everything is appended.** Mistakes are fixed with reversals.

4. **Every retry is safe.** Any request that moves money can be sent twice.
5. **Show the user the exact number.** Fees and totals are shown before confirmation and enforced at confirmation.
6. **Explain every state.** "Pending" always comes with a reason and a next step.

---

3. Users and personas

Persona	Who	Wants	Twofold gives them
Selam, everyday user	24, Addis Ababa, sends money to family in Hawassa	Fast, cheap transfers she can trust	Send by phone number, clear fees, instant receipts
Dawit, small merchant	Runs a café in Bole	Get paid without cash, see daily totals	Static QR code, payment notifications, daily summary
Hana, operations and compliance	Twofold staff	Review IDs, freeze bad accounts, answer "where is my money?"	Admin console: KYC queue, user lookup, transaction explorer, risk flags
Yonas, finance	Twofold staff	Fix mistakes without breaking the books	Maker-checker adjustments, ledger integrity dashboard, drift reports
Reviewer	Engineer at a fintech	Evidence that the author understands money	Tests, ADRs, load-test results, architecture docs

3.1 Roles

Customer app (`app.twofold.dev`):

Role	How you get it	Can
User	Sign up	Own wallet, send, request, top up, cash out (within tier limits)
Merchant	User upgrades with business details (KYC tier 2)	Everything a user can do, plus receive QR payments and see merchant reports

Admin console (`admin.twofold.dev`), staff only:

Role	Can
Support	Read users, wallets, and transactions; cannot change anything
Compliance	Review KYC, freeze and unfreeze wallets, resolve risk flags

Role	Can
Finance maker	Draft manual adjustments
Finance approver	Approve or reject adjustments drafted by *someone else*
Super admin	Manage staff accounts and settings

Staff accounts are a separate identity (`staff_users`), with mandatory TOTP MFA and a separate token audience. A staff token never works on the customer API, and a customer token never works on the admin API.

---

4. Scope

4.1 MVP (this plan)

Area	In MVP
Identity	Phone sign-up, OTP verification, password login, rotating refresh tokens, transaction PIN
KYC	Tiers 0–2, ID submission, manual review in admin
Wallets	One ETB wallet per user, created at sign-up; statuses active / frozen / closed
Ledger	Accounts, journal entries, postings, system accounts, immutability, reversals
Money movement	P2P transfer, top-up, withdrawal, money requests, merchant QR pay
Fees	Configurable fee rules, preview, enforced expected-fee check
Limits	Per-transaction, daily, monthly, and balance caps per tier
Async	Transactional outbox, background jobs, provider webhooks, notifications
Integrity	Nightly drift monitor, provider settlement reconciliation
Risk	Rule-based velocity and anomaly checks, holds, flags
Admin	Staff auth + MFA, KYC review, freeze, transaction explorer, maker-checker adjustments, audit log
Frontend	Customer web app (mobile-first PWA) and admin console
Ops	Metrics, traces, structured logs, load test, CI, containerized deploy

4.2 Later (named, not built)

Multi-currency wallets with FX, outbound merchant webhooks and a public merchant API, recurring payments, savings goals ("vaults"), Equb (rotating savings groups), USSD channel, native mobile app, Amharic UI, real provider integrations.

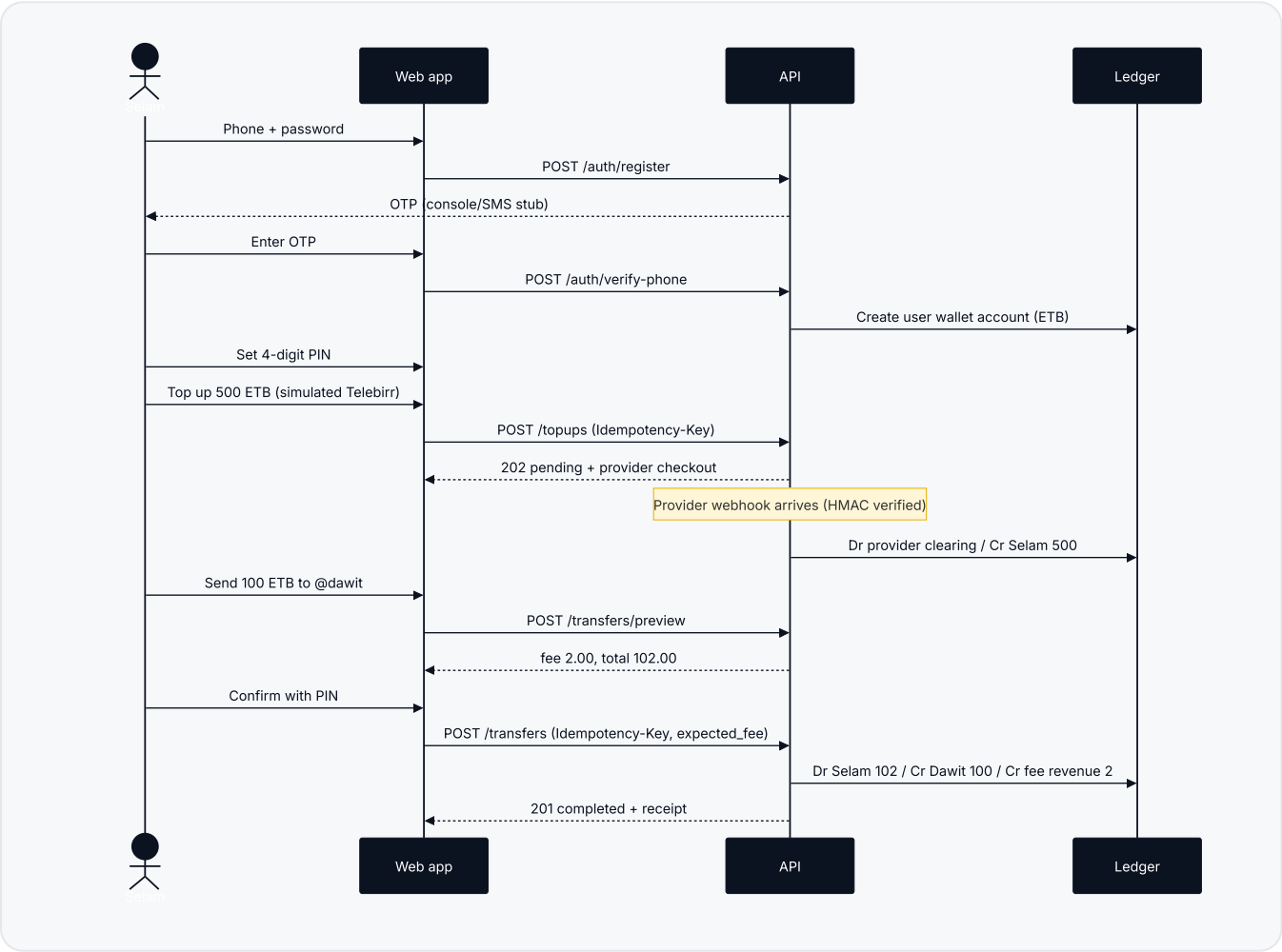
4.3 Non-goals

Real money, card issuing, credit, crypto, storing third-party bank or provider passwords, autonomous fraud *decisions* without a human on high-impact actions.

---

5. Core user journeys

5.1 Sign up to first transfer (target: under 3 minutes)



5.2 Every journey in the MVP

#	Journey	Happy path	Must also handle
J1	Sign up	Phone, OTP, password, PIN	Phone already registered, OTP expired, too many attempts
J2	Log in	Phone + password	Wrong password (generic error), locked account, new device
J3	Top up	Choose provider, amount, approve in provider, credited	Provider declines, callback duplicated, callback never arrives (expires)
J4	Send money	Recipient lookup, amount, fee preview, PIN, receipt	Insufficient funds, frozen wallet, limit exceeded, fee changed, double tap
J5	Request money	Pick contact, amount, note; recipient pays	Decline, cancel, expire after 7 days, pay with insufficient funds
J6	Merchant pay	Scan QR, amount (or fixed amount), PIN	Unknown or inactive merchant, tampered QR
J7	Cash out	Choose destination, amount, PIN, funds held, settled	Provider fails (funds return), stuck in processing
J8	History and receipts	Filter, open receipt, export CSV	Pending and failed items shown clearly
J9	KYC upgrade	Upload ID, wait for review, limits increase	Rejected with reason, resubmit
J10	Security	Change PIN, see sessions, log out a device	PIN locked after 5 failures

6. Functional requirements

Requirement IDs (`FR-…`) are referenced by the implementation plan.

6.1 Identity and access

ID	Requirement
FR-ID-1	Sign up with an Ethiopian phone number, normalized to E.164 (`+2519XXXXXXXX` or `+2517XXXXXXXX`) and a password (min 10 chars, checked against a breached-password list)
FR-ID-2	Phone verified with a 6-digit OTP: 5-minute expiry, 5 attempts, 60-second resend cooldown; stored hashed
FR-ID-3	Passwords hashed with argon2id (memory ≥ 64 MiB, parameters stored with the hash)

ID	Requirement
FR-ID-4	Login returns a 15-minute access JWT (`aud=twofold-app`) and a 30-day refresh token
FR-ID-5	Refresh tokens are stored hashed, rotated on every use, and **reuse of an old token revokes the whole session family**
FR-ID-6	Wrong phone and wrong password return the same error and take the same time
FR-ID-7	A 4-digit transaction PIN, hashed separately, is required to move money; 5 wrong PINs lock money movement for 30 minutes
FR-ID-8	Users can list active sessions and revoke any of them
FR-ID-9	Every user has a unique, changeable `@handle` (3–20 chars, lowercase, letters, digits, `_`)

6.2 KYC tiers

Tier	How to reach it	Purpose
0	Phone verified	Try the product with small amounts
1	Full name, date of birth, national ID (Fayda) number, ID photo, approved by compliance	Everyday use
2	Tier 1 + business name, TIN, trade license number, approved	Merchants

- FR-KYC-1: A submission moves through `submitted → approved | rejected(reason)` . Resubmission creates a new row; history is kept.
- FR-KYC-2: Approval updates the user's tier and therefore their limits immediately.
- FR-KYC-3: ID images are stored in object storage (local MinIO in development), never in the database, and are only viewable by compliance staff through short-lived signed URLs.

6.3 Wallets

- FR-WAL-1: Verifying a phone creates exactly one ETB wallet. A wallet is a ledger account plus user-facing metadata.
- FR-WAL-2: Statuses: `active` , `frozen` (cannot send or cash out; can still receive, unless the freeze is "full"), `closed` (no movement; balance must be zero to close).
- FR-WAL-3: The balance shown to users is `available` (balance minus holds). `ledger_balance` is visible on receipts and in admin.
- FR-WAL-4: A user can never read or act on another user's wallet. Every lookup is scoped by the authenticated user ID.

6.4 Peer-to-peer transfers

- FR-TRF-1: Recipient by phone or `@handle`. Lookup returns display name and masked phone (`+251 9•• ••• 123`) to confirm identity.
- FR-TRF-2: `POST /transfers/preview` returns amount, fee, total debit, and the recipient's display name. It moves no money.
- FR-TRF-3: `POST /transfers` requires: `Idempotency-Key`, transaction PIN, and `expected_fee_minor`. If the computed fee differs, return `409 fee_changed` and move nothing.
- FR-TRF-4: Checks run in this order, inside one database transaction: sender active → recipient active → not self → limits → risk → available balance ≥ amount + fee → post journal.
- FR-TRF-5: Success returns `201` with the transaction and receipt reference (`TF-7X3K-9QPA`).
- FR-TRF-6: Optional note, max 140 chars, stored as untrusted text and escaped on display.

6.5 Top-ups

- FR-TOP-1: User picks a provider (simulated `telebirr` or `chapa`) and an amount. The API creates a `pending` top-up and returns a provider checkout reference.
- FR-TOP-2: Money is credited only when a **verified** provider webhook says `succeeded`. The webhook signature is HMAC-SHA256 over `timestamp + "." + body`, with a 5-minute tolerance.
- FR-TOP-3: Webhooks are deduplicated by provider event ID. A duplicate returns `200` and does nothing.
- FR-TOP-4: A pending top-up with no webhook after 30 minutes is checked against the provider's status endpoint, then `expired` if still unknown.
- FR-TOP-5: Posting: Dr `provider_clearing:{provider}` / Cr user wallet.

6.6 Withdrawals (cash-out)

- FR-WDR-1: User chooses a saved destination (bank account or mobile-money number) and an amount, confirms with PIN.
- FR-WDR-2: On request, funds move from the wallet to `payouts_pending` immediately (the user cannot spend them twice), and a payout job is queued.
- FR-WDR-3: On provider success: Dr `payouts_pending` / Cr `provider_clearing:{provider}`. On failure: a reversal journal returns funds to the wallet.
- FR-WDR-4: The user always sees one of: processing, completed, or failed-and-refunded.

6.7 Money requests

- FR-REQ-1: A user requests an amount from another user with an optional note.
- FR-REQ-2: The payer sees it in an inbox and can pay (runs the transfer flow) or decline.
- FR-REQ-3: The requester can cancel while pending. Requests expire after 7 days.
- FR-REQ-4: Paying a request is idempotent per request: a request can be paid at most once (enforced by a unique constraint).

6.8 Merchant QR payments

- FR-MER-1: A tier-2 user can create a merchant profile (display name, category, logo) and gets a static QR.
- FR-MER-2: QR payload: `twofold://pay?m=<merchant_id>&a=<amount_minor?>&c=ETB&sig=<hmac>`. The signature stops tampering with fixed-amount codes.
- FR-MER-3: Merchant payments are transfers with `kind=merchant_payment` and the merchant fee rule (charged to the merchant, not the payer).
- FR-MER-4: Merchants see a daily summary: count, gross, fees, net.

6.9 Fees

- FR-FEE-1: Fee rules are data, not code: `kind`, `tier`, `flat_minor`, `rate_bps`, `min_minor`, `max_minor`, `effective_from`.
- FR-FEE-2: Fee = clamp(flat + amount × bps / 10,000, min, max), rounded **half-up to the nearest minor unit** with integer math only.
- FR-FEE-3: Illustrative defaults: P2P free under 100 ETB, else 0.5% (min 0.50, max 15.00 ETB); merchant payments 1% charged to the merchant; cash-out flat 5.00 ETB; top-up free.

6.10 History, receipts, statements

- FR-HIS-1: Paginated history (cursor based), newest first, filter by kind, status, date range, counterparty.
- FR-HIS-2: Every item has a receipt: reference, time (shown in Africa/Addis_Ababa, stored UTC), amount, fee, counterparty, status, and balance after.
- FR-HIS-3: Export CSV for a date range (max 12 months).

6.11 Notifications

- FR-NOT-1: In-app notifications for money received, top-up completed or failed, cash-out completed or failed, request received, KYC decision, security events (new login, PIN changed).
- FR-NOT-2: Email and SMS go through a `Sender` interface; development uses a console sender.
- FR-NOT-3: Notifications are produced from outbox events, never inline in the money transaction.

6.12 Audit

- FR-AUD-1: Append-only `audit_logs`: actor (user or staff), action, target, before/after summary, IP, user agent, request ID, time.
- FR-AUD-2: UPDATE and DELETE on `audit_logs` are blocked by a database trigger.
- FR-AUD-3: Covers sign-in, PIN changes, session revocation, KYC decisions, freezes, adjustments, staff logins, and settings changes.

---

7. The ledger

This is the most important section. Everything else is a client of the ledger.

7.1 Model

- **Account**: a bucket of value in one currency. Every user wallet is an account. The platform also has *system accounts*.
- **Journal entry**: one business event (a transfer, a top-up). It has two or more postings.
- **Posting**: one line: account, direction (`debit` or `credit`), positive amount in minor units.

The invariant: for every journal entry and every currency, `sum(debits) = sum(credits)`. A journal that does not balance is rejected at the database before commit.

7.2 Account types and normal balances

From the platform's point of view, money in a user's wallet is money the platform **owes** the user, so user wallets are **liabilities**.

Type	Normal balance	Balance formula	Examples
Asset	Debit	debits − credits	`provider_clearing:telebirr`, `provider_clearing:chapa`
Liability	Credit	credits − debits	every user wallet, `payouts_pending`
Revenue	Credit	credits − debits	`fee_revenue`
Expense	Debit	debits − credits	`promo_expense` (later)
Equity / suspense	Credit	credits − debits	`suspense` (unexplained provider money)

User wallet accounts have `allow_negative = false`, enforced by a check on the cached balance. System accounts may go negative when that is meaningful.

7.3 Postings for every flow

All amounts in ETB.

Flow	Debit	Credit
Top-up 500 via Telebirr	`provider_clearing:telebirr` 500	`wallet:selam` 500
P2P 100, fee 2	`wallet:selam` 102	`wallet:dawit` 100, `fee_revenue` 2
Merchant pay 300, merchant fee 3	`wallet:selam` 300, `wallet:cafe` 3	`wallet:cafe` 300, `fee_revenue` 3

Flow	Debit	Credit
Cash-out 200, fee 5 (request)	`wallet:selam` 205	`payouts_pending` 200, `fee_revenue` 5
Cash-out settles	`payouts_pending` 200	`provider_clearing:telebirr` 200
Cash-out fails (reversal)	`payouts_pending` 200, `fee_revenue` 5	`wallet:selam` 205
Manual adjustment (approved)	`suspense` or the target account	the other side, with a reason

> The merchant row shows why postings carry direction instead of a signed amount: the café is credited 300 and debited 3 in the same journal, and both lines stay readable.

7.4 Immutability

- `journal_entries` and `postings` are insert-only. A trigger raises an exception on UPDATE or DELETE.
- A correction is a **reversal**: a new journal with every posting's direction flipped and `reverses_journal_id` set. A journal can be reversed at most once (unique constraint).
- Reversals are only created by the system (failed payouts) or by an approved maker-checker adjustment.

7.5 Balances

- `accounts.balance_minor` and `accounts.version` are a **cache**, updated in the same database transaction as the postings that justify them.
- Each posting stores `balance_after_minor` and `account_version` for that account. That gives you a running balance in history without recomputing it.
- The drift monitor (§11.5) proves the cache equals the sum of postings, every night, for every account.

7.6 Money representation

- Amounts are `BIGINT` minor units (santim). `105000` = 1,050.00 ETB. In Go: `int64`. In JSON: an integer field ending in `_minor`, plus `currency`.
- Never `float`, never `numeric` for arithmetic, never a formatted string in the API.
- Overflow guard: reject any single amount above 10^{12} minor units.
- Formatting (`1,050.00 ETB`) happens only in the frontend.

---

8. Correctness under concurrency

8.1 The rule

Inside one database transaction (READ COMMITTED):

1. Collect every **user wallet** account ID the journal touches.
2. **Sort the IDs** and `SELECT … FOR UPDATE` them in that order.
3. Check balances and statuses on the locked rows.
4. Insert the journal and postings.
5. Update each account's cached balance with `WHERE id = $1 AND version = $2`, then bump the version.
6. Commit.

8.2 Why each part exists

Part	Prevents
`FOR UPDATE` row lock	Lost updates: two transfers reading the same starting balance
Sorted lock order	Deadlocks: A→B and B→A at the same time each holding one lock and waiting for the other
Version check	Silent corruption if a future code path forgets to lock; a mismatch is logged as an invariant violation
One database transaction	Partial writes: postings without a balance update, or the reverse

System accounts (`fee_revenue` , `provider_clearing:*`) are hot rows that every transfer touches. They are **not** locked. Their balance cache is refreshed by an aggregate job, and their postings are still written in the same transaction. Locking them would serialize the whole platform behind one row.

8.3 Proof

A concurrency test runs 100 goroutines doing random opposing transfers between 10 wallets against a real PostgreSQL instance. Expected: zero deadlocks, zero errors other than insufficient funds, total of all wallet balances unchanged, and the drift check passes.

---

9. Idempotency

Twofold closes a gap the reference project documents: there, a repeated key hits a unique constraint and returns `500` . Here, a repeated key returns the original response.

9.1 Contract

- Every endpoint that moves money or creates something requires an `Idempotency-Key` header (UUID v4, generated by the client once per user intent).
- Keys are scoped per user and kept for 24 hours.

9.2 Algorithm

```
hash = sha256(method + path + canonical JSON body)
INSERT INTO idempotency_keys (user_id, key, request_hash, state='in_progress')
  ON CONFLICT (user_id, key) DO NOTHING

if inserted:
    run the handler
    store status code + response body, state='completed'
else:
    existing = SELECT … WHERE user_id, key
    if existing.request_hash ≠ hash  → 422 idempotency_key_reused
    if existing.state = 'in_progress' → 409 request_in_progress (Retry-After: 1)
    if existing.state = 'completed'  → replay stored status + body
                                       header Idempotent-Replayed: true
```

9.3 Layers of defence

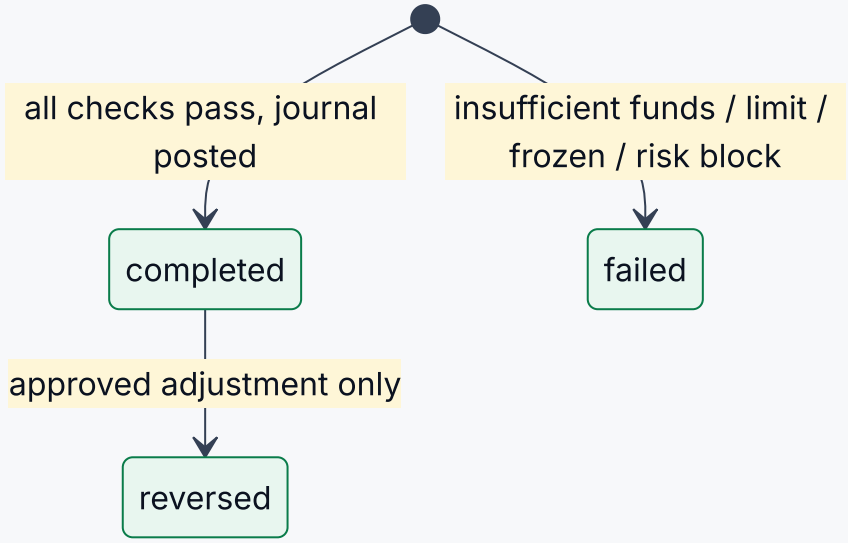
1. The idempotency table (above).
2. Domain uniqueness: a money request can be paid once; a provider event ID is processed once; a journal is reversed once.
3. Client behaviour: the confirm button disables after the first tap and reuses the same key on retry.

---

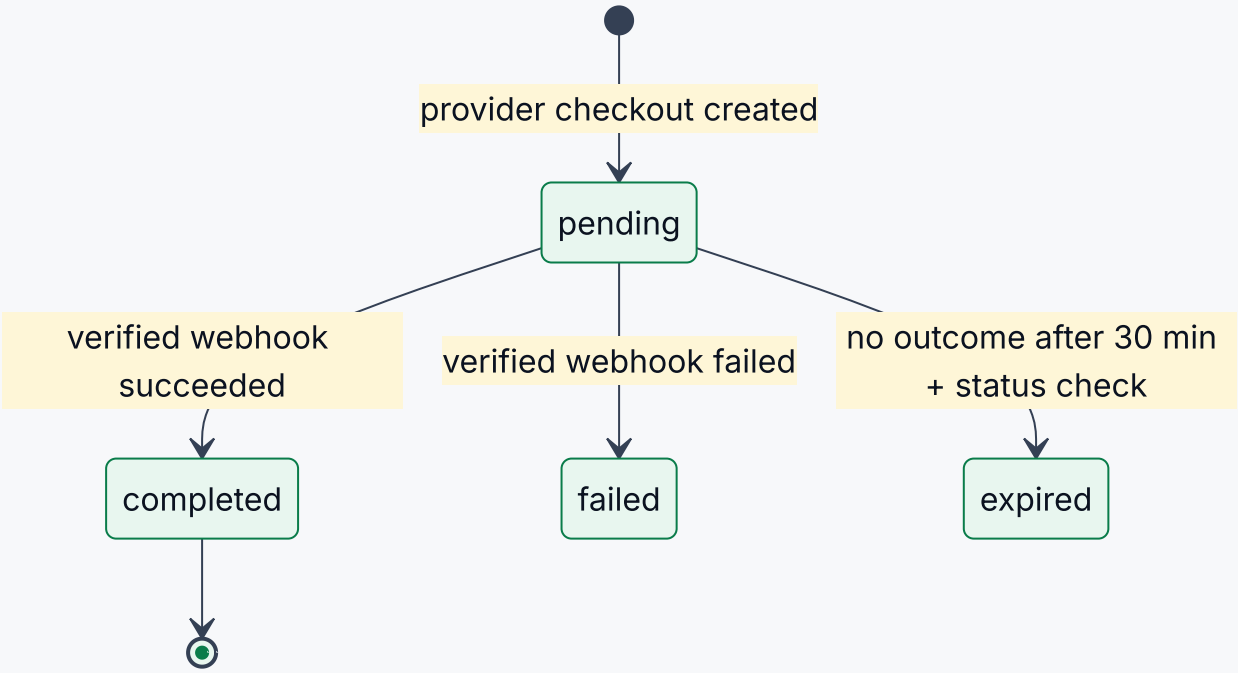
10. State machines

State changes are recorded as rows in `transaction_events`, never as silent overwrites.

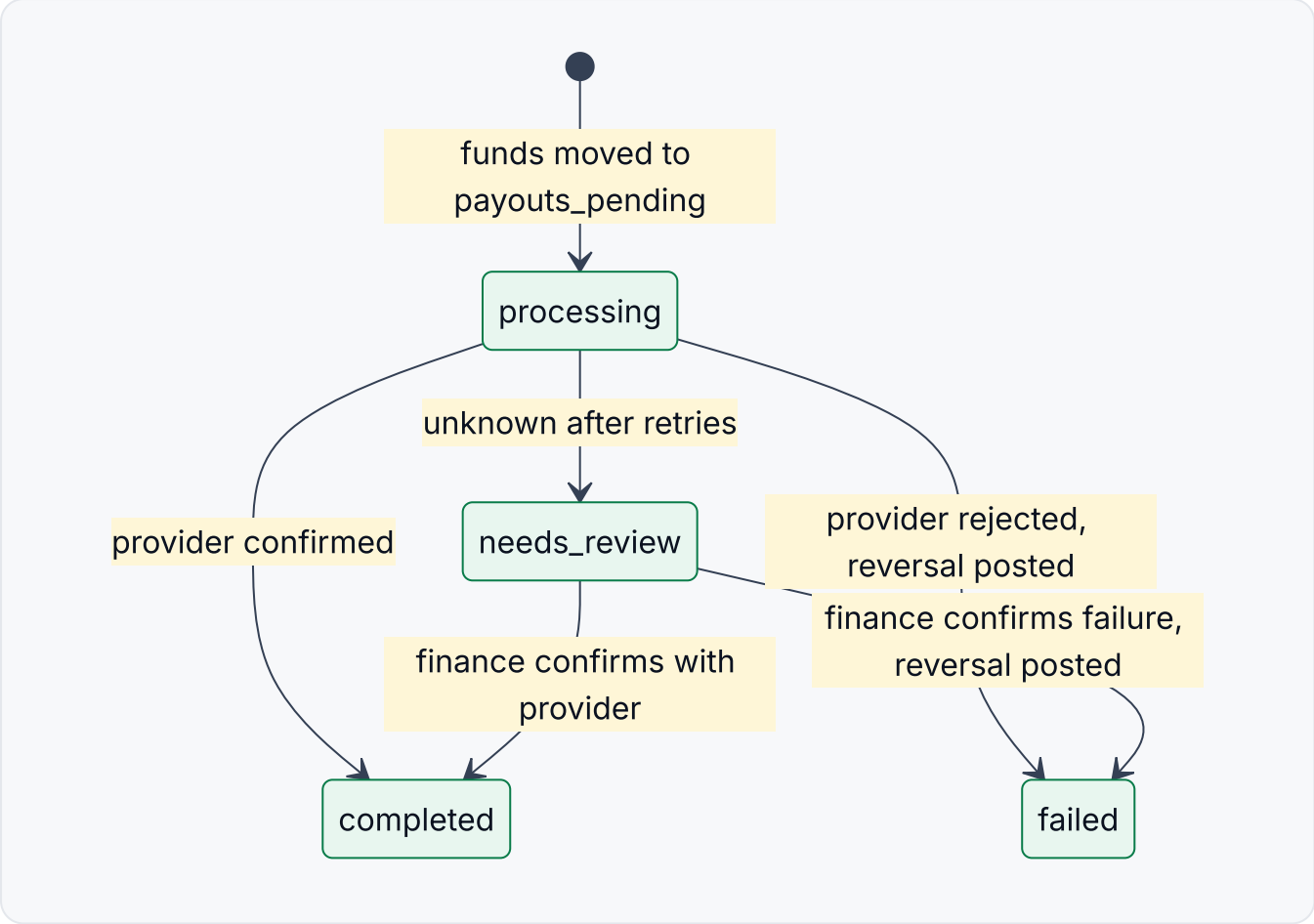
10.1 Transfer (synchronous)



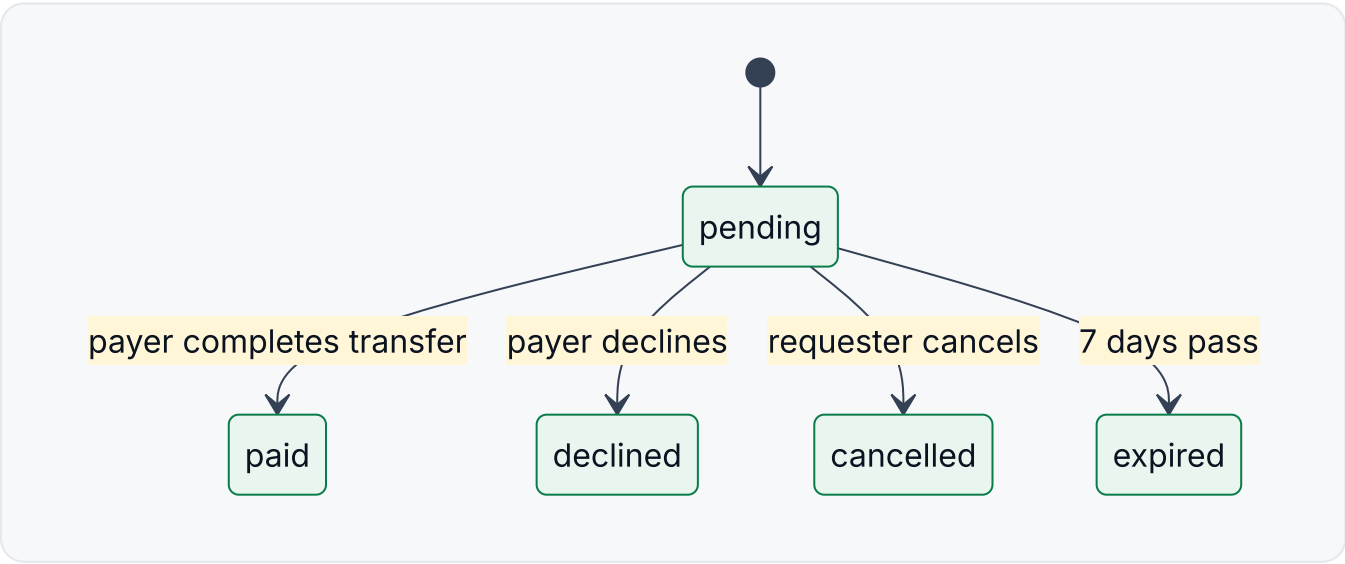
10.2 Top-up



10.3 Withdrawal



10.4 Money request



11. Asynchronous work: outbox, jobs, webhooks

11.1 Transactional outbox

When something happens that other parts of the system care about (`transfer.completed`, `topup.failed`), the event row is written to `outbox_events` **in the same database transaction** as the money movement. A dispatcher reads unsent rows and hands them to workers. If the process crashes after commit, the event is still there; if the transaction rolls back, the event never existed.

11.2 Job queue

Background jobs run on **River** (a PostgreSQL-backed queue), so there is no extra infrastructure. Jobs: dispatch outbox, send notification, execute payout, poll stale top-ups, expire requests, nightly drift check, settlement reconciliation, refresh system-account balances.

Jobs are idempotent and retried with exponential backoff. After max attempts they land in a dead-letter state visible in admin.

11.3 Inbound provider webhooks

`POST /webhooks/providers/{provider}`:

1. Verify the HMAC signature and timestamp. Reject with `401` before parsing anything else.
2. Insert `(provider, event_id)` into `webhook_events`. On conflict, return `200` (duplicate).
3. Apply the state transition in the same transaction.
4. Return `200` quickly. Heavy work goes to jobs.

11.4 Simulated providers

`internal/provider` defines one interface:

```
type Provider interface {
    Name() string
    CreateCollection(ctx context.Context, req CollectionRequest) (CollectionRef, error)
    CreatePayout(ctx context.Context, req PayoutRequest) (PayoutRef, error)
    GetStatus(ctx context.Context, ref string) (Status, error)
}
```

The simulator implements it and can be told to succeed, fail, delay, send duplicate webhooks, or never answer. Every failure path in §10 is testable on a laptop.

11.5 Integrity jobs

Job	Schedule	Checks	On mismatch
Drift monitor	Nightly + on demand	For every account: cached balance = sum of postings; for every journal: debits = credits	Critical alert, admin integrity dashboard turns red, row in `integrity_reports`
Settlement reconciliation	Daily per provider	Provider settlement file (simulated CSV) vs ledger postings on `provider_clearing:*`	Unmatched lines go to a review list; unexplained money goes to `suspense` only through an approved adjustment

This is where the RealLedger idea lives on: reconciliation is a feature of the wallet now.

---

12. Risk and limits

12.1 Limits (illustrative)

Limit (ETB)	Tier 0	Tier 1	Tier 2 (merchant)
Max balance	5,000	100,000	1,000,000
Per transaction	1,000	25,000	200,000
Daily outgoing	2,000	50,000	500,000
Monthly outgoing	10,000	300,000	3,000,000
Cash-out	Not allowed	Allowed	Allowed

- Limits live in a `tier_limits` table.
- Daily and monthly usage is computed from postings (per user, in Africa/Addis_Ababa days), not stored counters that can drift.
- An incoming payment that would exceed the recipient's max balance is rejected before anything is posted, with a message the sender understands.

12.2 Risk rules

Rules run inside the transfer flow (fast, synchronous) and return `allow`, `flag` (allow + create a flag for review), `hold` (do not move money; ask for review), or `block`.

Rule	Example trigger	Action
Velocity	More than 5 outgoing transfers in 10 minutes	flag
New recipients	More than 5 new recipients in 24 hours	flag

Rule	Example trigger	Action
Amount spike	Amount > 5× the user's 30-day median and > 2,000 ETB	hold
New device	First transfer from a device seen < 24 hours ago and amount > 1,000 ETB	hold
Frozen counterpart	Recipient wallet frozen	block

- Rules are code with thresholds in config, each with unit tests.
- User-facing wording is neutral: "This payment needs a quick review", never "fraud".
- Every decision is stored (`risk_decisions`) with the rule versions that produced it, so a decision can be explained later.

---

13. Admin console

Area	Features	Who
Sign-in	Email + password + TOTP, 30-minute idle timeout, optional IP allowlist	All staff
Overview	Today's volume, counts by status, failed and stuck items, drift status	All staff
Users	Search by phone, handle, name; profile, tier, sessions, wallets	Support+
KYC queue	Submitted items oldest first; view ID via signed URL; approve or reject with reason	Compliance
Wallets	Freeze (send-only or full) and unfreeze with a reason	Compliance
Transactions	Search by reference, user, provider ref; view journal and postings	Support+
Risk flags	Queue, resolve as cleared / confirmed (freeze), notes	Compliance
Adjustments	Maker drafts (accounts, amounts, reason, evidence); a *different* approver approves; then it posts	Finance
Integrity	Latest drift and reconciliation reports, unmatched lines	Finance
Audit	Filterable audit log, export	Super admin, compliance
Dead letters	Failed jobs with retry button	Super admin

Maker-checker is enforced in the database: `CHECK (approved_by IS NULL OR approved_by <> created_by)` .

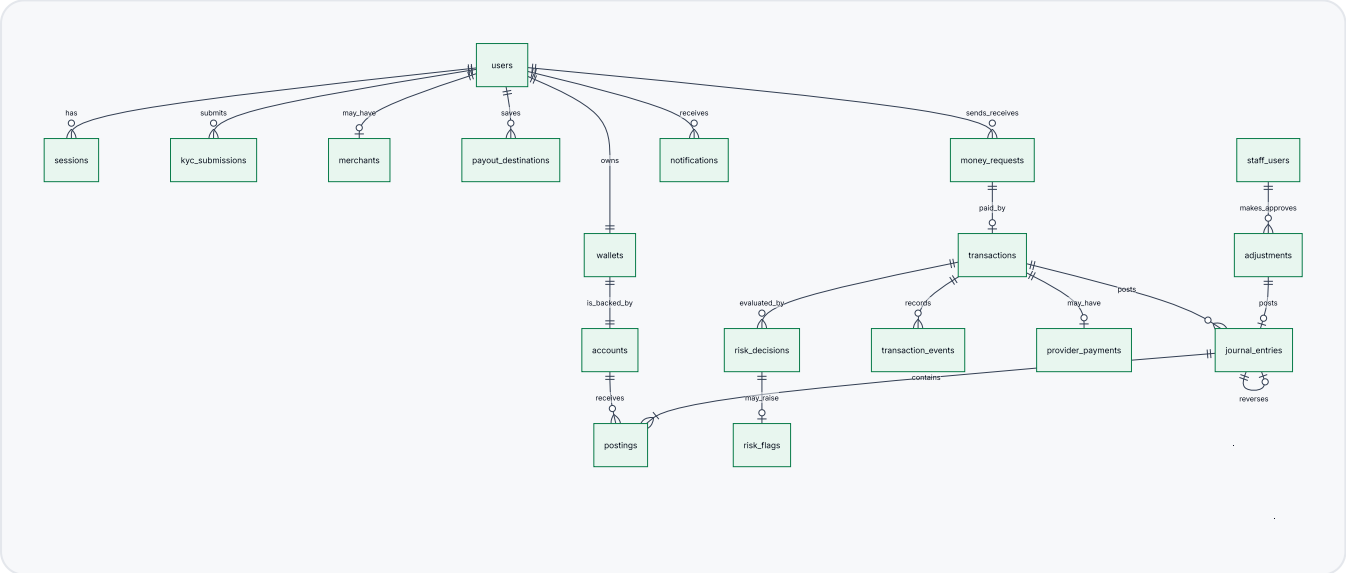
---

14. Data model

14.1 Conventions

Topic	Rule
IDs	`uuid` primary keys (`gen_random_uuid()`); public references (`TF-…`) are separate, human-friendly, unique
Time	`timestamptz` , stored in UTC; display in Africa/Addis_Ababa
Money	`bigint` `*_minor` + `currency char(3)`
Names	`snake_case` , plural tables, `<singular>_id` foreign keys, `is_` booleans, `*_at` timestamps
Enums	`text` + `CHECK` constraint (easy to evolve in migrations)
Deletes	Business records are never hard-deleted; status changes or `deleted_at`
Append-only	`journal_entries` , `postings` , `audit_logs` , `transaction_events` , `webhook_events`

14.2 Entity relationships



14.3 Tables

Table	Key columns	Notes
`users`	id, phone_e164 (unique), handle (unique), display_name, password_hash, pin_hash, pin_failed_count, pin_locked_until, kyc_tier, phone_verified_at, status	
`otp_codes`	id, phone_e164, purpose, code_hash, attempts, expires_at, consumed_at	

Table	Key columns	Notes
`sessions`	id, user_id, family_id, refresh_hash, device_id, user_agent, ip, expires_at, revoked_at, replaced_by	Rotation + reuse detection
`kyc_submissions`	id, user_id, target_tier, data (jsonb), document_keys, status, reviewed_by, reviewed_at, reason	
`accounts`	id, code (unique), type, normal_balance, currency, allow_negative, balance_minor, version, owner_user_id (nullable)	Ledger accounts
`wallets`	id, user_id (unique per currency), account_id (unique), status, freeze_mode, frozen_reason	User-facing
`journal_entries`	id, transaction_id, kind, reverses_journal_id (unique, nullable), description, created_at	Insert-only
`postings`	id, journal_id, account_id, direction, amount_minor (> 0), currency, balance_after_minor, account_version	Insert-only
`transactions`	id, reference (unique), kind, status, initiator_user_id, from_wallet_id, to_wallet_id, amount_minor, fee_minor, currency, note, idempotency_key, created_at, completed_at	Business record users see
`transaction_events`	id, transaction_id, from_status, to_status, reason, actor, created_at	
`provider_payments`	id, transaction_id, provider, direction (collection/payout), provider_ref (unique per provider), status, raw (jsonb)	
`webhook_events`	provider, event_id (pk together), received_at, payload_hash	Dedupe
`payout_destinations`	id, user_id, provider, account_identifier (masked + encrypted), label	
`money_requests`	id, requester_id, payer_id, amount_minor, currency, note, status, paid_transaction_id (unique), expires_at	
`merchants`	id, user_id (unique), display_name, category, qr_secret, status	
`fee_rules`	id, kind, tier, flat_minor, rate_bps, min_minor, max_minor, effective_from	
`tier_limits`	tier, max_balance_minor, per_txn_minor, daily_minor, monthly_minor, can_withdraw	
`idempotency_keys`	user_id, key (pk together), request_hash, state, response_code, response_body, expires_at	
`outbox_events`	id, topic, payload (jsonb), created_at, dispatched_at	
`notifications`	id, user_id, kind, title, body, read_at, created_at	
`risk_decisions`	id, transaction_id, outcome, rules (jsonb), created_at	

Table	Key columns	Notes
`risk_flags`	id, user_id, decision_id, status, resolved_by, resolution, notes	
`staff_users`	id, email, password_hash, totp_secret (encrypted), role, status	Separate identity
`adjustments`	id, created_by, approved_by, status, reason, evidence_url, lines (jsonb), journal_id	Maker-checker
`audit_logs`	id, actor_type, actor_id, action, target_type, target_id, before, after, ip, user_agent, request_id, created_at	Append-only
`integrity_reports`	id, kind, started_at, finished_at, accounts_checked, mismatches (jsonb), status	

15. API overview

Full contract: `docs/openapi.yaml` (written on Day 21 and checked in CI after that). Rules:

- Base path `/v1` for customers, `/admin/v1` for staff.
- JSON only. Money fields are integers ending in `_minor`, plus `currency`.
- Errors use RFC 9457 problem details:

```
{
  "type": "https://twofold.dev/errors/insufficient-funds",
  "title": "Insufficient funds",
  "status": 422,
  "detail": "Available balance is 80.00 ETB; this transfer needs 102.00 ETB.",
  "code": "insufficient_funds",
  "request_id": "req_01HV…"
}
```

15.1 Customer API

Method	Path	Purpose	Idempotency-Key
POST	`/v1/auth/register`	Start sign-up, send OTP	
POST	`/v1/auth/verify-phone`	Verify OTP, create wallet	
POST	`/v1/auth/login`	Tokens	
POST	`/v1/auth/refresh`	Rotate refresh token	
POST	`/v1/auth/logout`	Revoke current session	

Method	Path	Purpose	Idempotency-Key
GET	`/v1/me`	Profile, tier, limits usage	
PUT	`/v1/me/pin`	Set or change PIN	
GET / DELETE	`/v1/me/sessions[/{id}]`	List, revoke sessions	
GET	`/v1/wallets`	My wallets with available and ledger balance	
GET	`/v1/recipients/lookup?q=`	Find user by phone or handle (masked)	
POST	`/v1/transfers/preview`	Fee and total	
POST	`/v1/transfers`	Send money	✅
POST	`/v1/topups`	Start top-up	✅
GET	`/v1/topups/{id}`	Top-up status	
POST	`/v1/withdrawals`	Start cash-out	✅
GET / POST	`/v1/payout-destinations`	Saved destinations	✅ (POST)
GET	`/v1/transactions`	History (cursor, filters)	
GET	`/v1/transactions/{reference}`	Receipt	
GET	`/v1/transactions/export.csv`	CSV export	
POST	`/v1/requests`	Request money	✅
GET	`/v1/requests?box=incoming|outgoing`	Inbox	
POST	`/v1/requests/{id}/pay`	Pay a request	✅
POST	`/v1/requests/{id}/decline` · `/cancel`	Decline or cancel	
POST	`/v1/merchants`	Become a merchant	✅
GET	`/v1/merchants/me/qr`	QR payload	
POST	`/v1/merchant-payments`	Pay by QR	✅
POST	`/v1/kyc/submissions`	Submit KYC	✅
GET	`/v1/notifications`	Notifications	

Method	Path	Purpose	Idempotency-Key
POST	`/webhooks/providers/{provider}`	Provider callbacks (HMAC, no JWT)	
GET	`/health` · `/ready` · `/metrics`	Ops	

15.2 Admin API

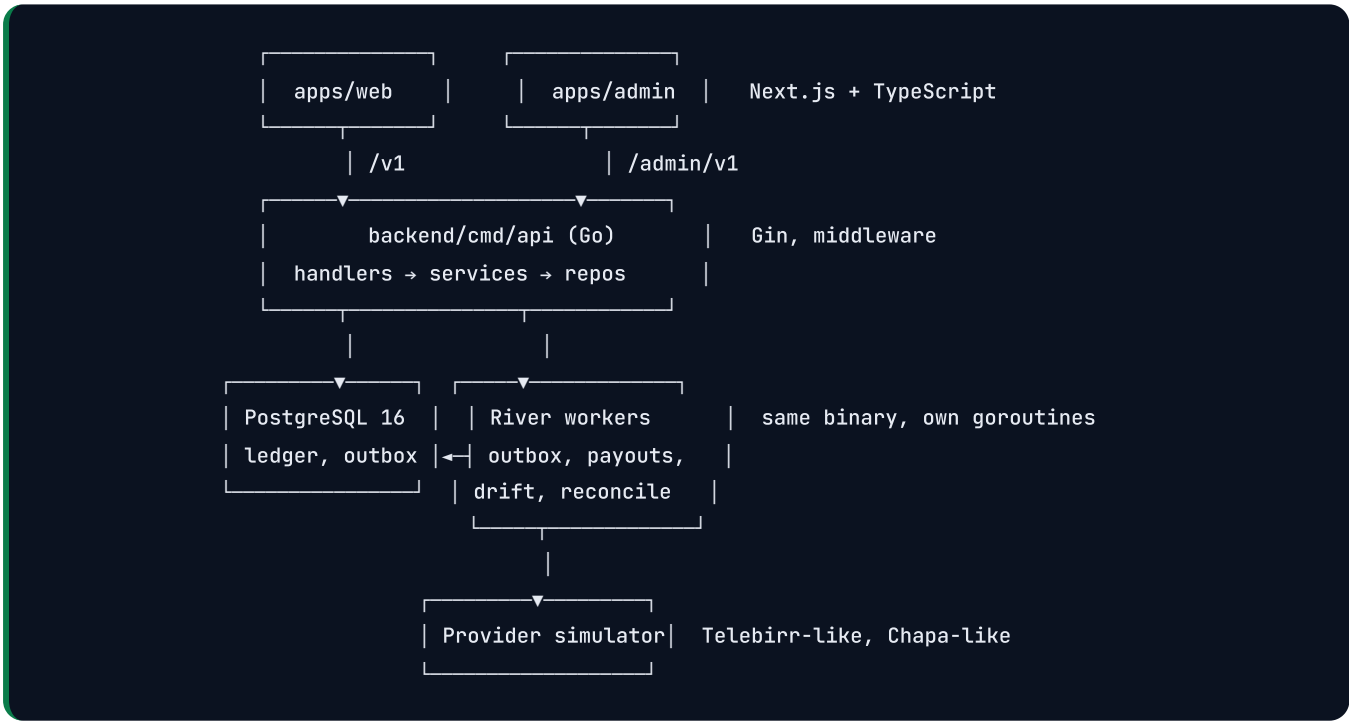
`/admin/v1/auth/*`, `/users`, `/kyc/submissions`, `/wallets/{id}/freeze`, `/transactions`, `/journals/{id}`, `/risk-flags`, `/adjustments` (+ `/approve`, `/reject`), `/integrity/reports`, `/audit-logs`, `/jobs/dead`.

---

16. Architecture

16.1 Shape

A **modular monolith**: one Go binary, clear package boundaries, one PostgreSQL database. Splitting into services is a later decision with a reason, not a starting point.



16.2 Backend layout (target, reshaped on Day 2)

```
backend/
├── cmd/api/                composition root: config, wiring, server, workers
├── internal/
│   ├── platform/           config, database, httpx (errors, middleware), clock, ids
│   ├── auth/               register, OTP, login, sessions, PIN
│   ├── user/               profile, handle, KYC
│   ├── ledger/             accounts, journals, postings, balances  ← the core
│   ├── wallet/             wallet status, available balance, holds
│   ├── transfer/           P2P, merchant payments, preview
│   ├── fee/                fee rules and calculation
│   ├── limit/              tier limits and usage
│   ├── risk/               rules engine and decisions
│   ├── funding/            top-ups and withdrawals
│   ├── provider/           provider interface + simulator
│   ├── request/            money requests
│   ├── merchant/           merchant profiles and QR
│   ├── idempotency/        middleware and store
│   ├── outbox/             outbox writer and dispatcher
│   ├── notification/       senders and in-app notifications
│   ├── integrity/          drift monitor and settlement reconciliation
│   ├── audit/              append-only audit log
│   └── admin/              staff auth, MFA, admin handlers
├── migrations/             goose SQL migrations
├── queries/                sqlc query files
└── tests/                  integration, concurrency, authorization suites
```

Dependency rule: handlers depend on services, services depend on interfaces, repositories implement them. `ledger` depends on nothing but `platform`. Nothing outside `ledger` writes to `postings`.

16.3 Stack

Layer	Choice	Why
Language	Go 1.25	Simple concurrency, strong standard library, common in payments
HTTP	Gin	Already in the repo; well known
Database	PostgreSQL 16	Transactions, row locks, constraints, triggers
Access	pgx + sqlc	Type-safe SQL without an ORM hiding the queries
Migrations	goose	Plain SQL, up and down
Jobs	River	Postgres-backed queue, transactional enqueue

Layer	Choice	Why
Auth	argon2id, JWT (EdDSA), TOTP	Modern defaults
Contract	OpenAPI 3.1 → TypeScript client	Frontend never guesses a shape
Frontend	Next.js (App Router), TypeScript, Tailwind, shadcn/ui, TanStack Query, React Hook Form + Zod	Fast to build, accessible primitives
Observability	slog JSON logs, Prometheus, OpenTelemetry, Grafana	Industry standard
Testing	Go testing, rapid (property tests), testcontainers-go, Playwright, k6	Each layer covered
Delivery	Docker, GitHub Actions, Fly.io or Render	Free-tier friendly

Architecture decisions are recorded as ADRs in `docs/adr/` (for example `0001-double-entry-with-direction.md`, `0002-lock-ordering.md`, `0003-idempotency-replay.md`).

---

17. Non-functional requirements

Category	Target
Correctness	Zero drift across load tests and the concurrency suite. Any drift is a release blocker.
Latency	P2P transfer p95 < 150 ms, p99 < 400 ms at 200 requests/second on a 2-vCPU instance
Throughput	500 transfers/second sustained in a load test without errors other than business rejections
Availability (target)	99.9% for the API in a hosted demo
Durability	Daily backups, point-in-time recovery enabled, one documented restore drill
Data retention	Ledger, transactions, and audit kept indefinitely; OTPs 24 hours; idempotency keys 24 hours
Accessibility	WCAG 2.2 AA for the web apps: contrast, keyboard paths, screen-reader labels on amounts
Performance (web)	Home screen LCP < 2.5 s on a mid-range phone over 3G Fast
Localization	English at launch; all copy in message files ready for Amharic; Ethiopic font loaded

---

18. Security and privacy

Threat	Control
Credential stuffing	Rate limits by IP and phone, progressive delays, breached-password check
Stolen access token	15-minute lifetime, audience check, EdDSA signatures, key rotation via `kid`
Stolen refresh token	Rotation with reuse detection that revokes the whole family
Unauthorized money movement	Transaction PIN with lockout; risk holds on new devices
Broken object-level authorization (BOLA)	Every query scoped by authenticated user ID; a test suite that tries every route with another user's IDs
Replay / double submit	Idempotency keys; domain unique constraints
Forged provider webhook	HMAC verification, timestamp tolerance, event-ID dedupe
Tampered QR	Signed QR payloads
Insider abuse	Separate staff identity, MFA, maker-checker, append-only audit
Injection	sqlc parameterized queries only; notes escaped on render; strict CSP on web
Secrets in git	`.env` ignored, gitleaks in CI, secrets in the host's secret store
Personal data exposure	Phone masked in lookups; ID images in private storage with signed URLs; destinations encrypted at rest; PII never in logs

Privacy follows Ethiopia's Personal Data Protection Proclamation No. 1321/2024: collect only what each tier needs, show a privacy notice at sign-up, support data export, and record where data is hosted.

---

19. Observability

- **Logs:** JSON via `slog`; every line carries `request_id`, `user_id` (when known), and `trace_id`. No phone numbers, PINs, tokens, or ID numbers in logs.
- **Metrics (Prometheus):** request rate, errors, and latency per route; `twofold_money_movements_total{kind,outcome}`; `twofold_ledger_drift_accounts` (must be 0); job queue depth and failures; webhook verification failures; risk outcomes.
- **Traces (OpenTelemetry):** HTTP → service → SQL spans, plus job spans linked to the request that enqueued them.
- **Dashboards:** a committed Grafana dashboard JSON with golden signals and money metrics.
- **Alerts (documented):** drift > 0, payout stuck > 1 hour, webhook failure spike, job dead letters > 0.

20. Testing strategy

Layer	Tool	What it proves
Unit	Go `testing`, table tests	Fee math, limits, state transitions, risk rules
Property	`pgregory.net/rapid`	For any random sequence of operations, every journal balances and money is conserved
Integration	testcontainers-go + PostgreSQL 16	Repositories, triggers, constraints, migrations up and down
Concurrency	100 goroutines, opposing transfers	No deadlocks, no lost updates, zero drift
Authorization	Route-by-route BOLA suite	User A gets 404 for every resource owned by user B; customer tokens rejected on admin API
Contract	OpenAPI validation in CI	Responses match the spec; generated client is up to date
End-to-end	Playwright	Sign up → top up → send → receipt; cash-out failure refund
Load	k6	Latency and throughput targets in §17
Chaos-lite	Kill the API mid-transfer, duplicate webhooks, delayed provider	No partial writes, no double credits

Coverage target: `ledger`, `transfer`, `funding`, and `idempotency` ≥ 85% statement coverage. The rest is judged by the suites above, not a number.

21. Frontend requirements

Visual language, colours, and components are defined in design-system.md.

21.1 Customer app (`apps/web`)

Screen	Key requirements
Welcome / sign up / OTP / PIN setup	4 steps max; OTP auto-advance; clear errors
Home	Balance card (available, with a hide toggle), quick actions (Send, Request, Top up, Cash out, Scan), recent activity, pending items banner
Send	Recipient search (phone or handle), masked confirmation, numeric keypad, live fee and total, PIN sheet, success receipt with share

Screen	Key requirements
Request	Contact, amount, note; incoming and outgoing tabs
Top up	Provider choice, amount, pending screen that polls, success or failure state
Cash out	Destination picker, amount, fee, PIN, processing timeline
Scan / My QR	Camera scanner; merchants show their QR
Activity	Infinite list, filters, receipt detail, CSV export
Profile and security	Handle, tier and limits usage bars, KYC upload, PIN change, sessions, logout

Rules:

- Every money-moving button generates the `Idempotency-Key` when the user *starts* the flow and reuses it on retry.
- Amounts are entered as strings and converted to minor units with integer math; never `parseFloat` for money.
- Optimistic UI is not used for money. The UI shows a confirming state until the server answers.
- Works on a 360 px wide screen first; keyboard and screen-reader paths for every flow.

21.2 Admin console (`apps/admin`)

Dense, table-first layout. Every destructive or money action requires a reason and shows a confirm dialog with the exact effect. Covered by §13.

---

22. Success metrics

22.1 Product metrics (if run with real users)

- Time from sign-up to first transfer under 3 minutes.
- Transfer success rate (excluding insufficient funds) above 99.5%.
- Top-ups stuck in pending more than 30 minutes: 0.

22.2 Engineering bar (portfolio)

Metric	Target
Drift across all suites and load tests	0 accounts
Concurrency suite	100 goroutines, 0 deadlocks
BOLA suite	Every customer route covered

Metric	Target
Load test	Targets in §17 met, results in `docs/perf.md`
ADRs	At least 6
Demo	Hosted URL, seeded demo users, 3-minute video
README	Reads as a case study: problem, design, proof, trade-offs

23. Risks and mitigations

Risk	Mitigation
Scope is too large for 7 weeks	Each week ends with something demoable; weeks 6–7 features are cut first (QR, risk UI)
Ledger bugs discovered late	Ledger and property tests land in week 2, before any feature uses them
Hot system accounts slow everything	Not locked; balances refreshed by aggregate job (§8.2)
Frontend drifts from the API	Generated client from OpenAPI; CI fails on drift
"Copied project" perception	Original code, credited inspiration, ADRs explaining every choice in your own words
Burnout	Sunday is review and testing, not new features; the plan allows cutting

24. What this project teaches

Each area maps to questions fintech interviewers actually ask.

Area	You will be able to explain	Days
Double-entry accounting	Why wallets are liabilities, normal balances, why postings carry direction	9–11
Concurrency	Lost updates, `SELECT FOR UPDATE`, deadlock prevention by lock ordering, optimistic versioning	15–16
Idempotency	Why retries are dangerous, key + hash + replay, domain unique constraints	13
Distributed consistency	Transactional outbox, at-least-once delivery, idempotent consumers	25–26

Area	You will be able to explain	Days
Async payments	Pending states, webhooks, signature verification, timeouts and polling	22–24
Reconciliation	Drift detection, settlement matching, suspense accounts	27
Security	Token rotation, BOLA, PIN lockouts, HMAC, MFA, maker-checker	5–8, 28, 39, 42
Testing money	Property-based tests, concurrency tests, contract tests	14, 16, 21
Operating a system	Metrics, traces, load testing, runbooks	45–48
Product thinking	Fee transparency, clear pending states, accessible money UI	29–38

25. Credits and prior art

- **Lakipay Wallet Service** by Muhammed Shamsadin (github.com/Muhammed-shamsadin/wallet-service). Ideas that shaped Twofold's design: deterministic lock ordering, combining row locks with version checks, a double-entry entries table behind a cached balance, and separate liveness and readiness probes. That repository has no license, so **no code is copied**. Twofold re-implements these ideas from scratch and addresses limitations the Lakipay README lists: idempotency replay instead of `500`, a scheduled drift monitor, asynchronous provider flows, and fees.
- **RealLedger** (this repository's previous product): ledger and reconciliation design.
- General references: Modern Treasury's writing on ledgers, Stripe's idempotency design, the PostgreSQL documentation on explicit locking, and River's transactional enqueue.

26. Glossary

Term	Meaning
Minor unit	Smallest currency unit as an integer; 1 ETB = 100 santim
Account	A ledger bucket of value in one currency
Journal entry	One business event, made of balanced postings
Posting	One debit or credit line against one account
Normal balance	The side (debit or credit) that increases an account
Clearing account	An asset account holding money in transit at a provider

Term	Meaning
Suspense	Where unexplained money waits until a human classifies it
Reversal	A new journal that cancels an earlier one by flipping every posting
Drift	A cached balance that no longer equals the sum of its postings
Idempotency key	A client-generated ID that makes a retry return the first result
Outbox	A table of events written in the same transaction as the change that caused them
BOLA	Broken object-level authorization: reading or changing someone else's object by guessing its ID
Maker-checker	Two different people must create and approve a sensitive action
KYC	Know your customer: identity verification tiers